

Your grade: 100%

Your latest: 100% • Your highest: 100% • To pass you need at least 80%. We keep your highest score.

Next item →

1. Which is an example of a scenario that is NOT parallel processing?

1 / 1 point

- ☐ Running a model simulation on multiple processors of climate fluctuations during the month of August from 1980-2000
- ☐ Driving down the street and noticing the smells, the temperature, the other cars around you simultaneously
- ☐ You and your friends preparing dinner - you cook the water, someone else prepares the sauce, another person cuts the vegetables, and yet another sets the table
- ☒ Getting ready to go for a run - you put on your shoes, tie them, stretch, and head out the door

👏 Nice work

This is a serial process because you do all these activities in succession.

See videos on serial and parallel processing review

2. You are running a program in serial, and decide to run it in parallel on your HPC system. You notice that your job is suddenly running 10% slower now than it was when it was running serially. Without any knowledge of the specifics of how you've done this, what are some possible reasons why the script is now running slower?

1 / 1 point

- ☐ Your program is not actually parallelizable
- ☒ The additional communication time between processes is outweighing any benefits from the parallelization
- ☐ It takes a while to turn off the cores once you've finished using them
- ☐ All of the above

👏 Nice work

This is an example of overhead, and can potentially cause a job to run slower than when running it serially in certain situations.

See videos on serial and parallel processing review

3. You have a task that you would like to run, using 20 nodes but only two cores on each node (each node has 10 cores). You are running code that needs to synchronize at certain points. Is there need for communication between the cores or nodes, and how?

1 / 1 point

- ☐ The cores will need to communicate, but the nodes will not.
- ☐ The nodes will need to communicate, but the cores will not.
- ☒ Yes; the tasks running on the cores on one node will communicate using a shared memory protocol, such as OpenMP. The nodes will communicate using a distributed message passing protocol over a network, such as MPI. MPI could also be used to communicate between cores.
- ☐ No; the tasks running on the cores and the nodes will not need to communicate, as the data will be run separately from each other.

👏 Nice work

All the cores on all the nodes, and among a single node, will need to pass the data that has been ran between them in order to complete the tasks for the entire program to finish.

See video on parallel memory models to review

4. What does the following example best describe? Someone entered in all the names of registered voters in lower case in a text file. The names need to be put in upper case. There are over 10 million entries.

1 / 1 point

- ☐ High Throughput Computing – this is not a communication intensive job, and so the names can be distributed amongst many cores.
- ☐ Task parallelism – the tasks to be completed are spread out among the cores/nodes while the data remains on one, and the operation of making the names upper case is performed on the same data.
- ☐ Data parallelism – the data is spread out among cores/nodes, and the same task (the operation of making the names upper case) is performed on the different data values.
- ☒ Both a) and c)

👏 Nice work

Both situations could apply to this task. Because you are performing the same task repeatedly, it makes sense to parallelize the data. Also, because the job does not require a lot of computational power or memory, it qualifies as a high throughput job as well.

See videos on data and task parallelism and high throughput computing to review

5. Let's say you want to do some timing within your program. What might be good areas of your code to time?

1 / 1 point

- ☐ A loop
- ☐ Comparison between two different algorithms doing the same task
- ☐ Input/Output sections, like writing out log files or restart files
- ☒ All of the above

👏 Nice work